



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики**

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

по дисциплине «Технологии и инструментарий анализа больших данных»

Лекции №1–2

Ответы на контрольные вопросы

Студент группы

ИНБО-12-23, Губарев С.А.

(подпись)

Преподаватель

Воронцов Ю.А.

(подпись)

Москва 2026 г.

Содержание

Назначение работы.....	3
Лекция №1. Подробные ответы на контрольные вопросы	3
Лекция №2. Подробные ответы на контрольные вопросы	9
Источники	15

Назначение работы

Документ содержит ответы на все 16 контрольных вопросов двух предоставленных лекций: архитектура больших данных, масштабирование и аналитические слои; SQL, оконные функции, планы выполнения и модели NoSQL. Нумерация вопросов соответствует страницам 70 первой лекции и 107 второй.

Лекция №1. Подробные ответы на контрольные вопросы

Тема: «Большие данные и современные архитектуры аналитических систем».

Вопросы из лекции №1, страница 70. Примеры ниже поясняют материал лекции.

1. Почему Big Data нельзя корректно определить только через объём данных?

Простыми словами. Данные становятся «большими», когда имеющаяся система не справляется с их хранением и обработкой в нужные сроки. Универсальной границы в гигабайтах или терабайтах нет: один и тот же набор может быть сложным для ноутбука и обычным для мощного сервера.

Как это работает. В лекции выделены шесть характеристик: Volume — объём, Velocity — скорость поступления, Variety — разнообразие форматов, Veracity — достоверность, Value — полезность результата, Variability — изменчивость данных. Например, небольшой поток событий может требовать немедленной реакции, а большой архив — допускать многочасовую обработку. Поэтому оценивают не только размер, но и доступную память, время обработки, пропускную способность, качество и требования к результату. Превышение объёма оперативной памяти — сигнал проверить архитектуру, но само по себе не означает, что обязательно нужен кластер: некоторые задачи можно выполнять порциями на одном узле.

Пример и вывод. Интернет-магазину нужно пересчитать продажи за год и обнаруживать подозрительную активность сразу после события. Для первого

задания допустим ночной расчёт, для второго важна малая задержка. Даже при меньшем объёме второе задание может быть сложнее технически. Big Data — это соотношение данных, ресурсов и требований задачи, а не просто большой файл.

2. Чем вертикальное масштабирование отличается от горизонтального?

Простыми словами. Вертикальное масштабирование — сделать один компьютер мощнее. Горизонтальное — подключить несколько компьютеров и распределить между ними работу.

Как это работает. При вертикальном подходе увеличивают оперативную память, число процессорных ядер, скорость дисков и сети одного узла. Это обычно проще в настройке, но возможности оборудования ограничены, а отказ единственного узла останавливает работу. При горизонтальном подходе данные и вычисления распределяют по узлам кластера. Суммарные ресурсы растут, однако появляются сетевые передачи, согласование состояния, балансировка нагрузки и обработка отказов. Удвоение числа узлов не гарантирует удвоения скорости: часть времени уходит на координацию и обмен данными.

Пример и вывод. Увеличить память сервера с 32 до 128 ГБ — вертикальное масштабирование. Разделить обработку истории продаж между четырьмя серверами — горизонтальное. Первый вариант удобен, пока одного узла достаточно; второй полезен, когда нагрузка допускает распределение и выходит за разумные ограничения одного сервера. Само добавление серверов не создаёт отказоустойчивость — её надо обеспечить архитектурой.

3. Зачем нужны partitioning и replication?

Простыми словами. Partitioning — разделить данные на части. Replication — создать несколько копий данных. Разделение помогает распределить объём и обработку, а копии помогают переживать отказы.

Как это работает. При partitioning записи распределяют, например, по месяцам, регионам или хешу идентификатора. Части можно обрабатывать

параллельно, а при подходящем условии запроса читать только нужные части. Разбиение возможно и внутри одного сервера; распределение частей по разным узлам часто называют шардированием. При replication одинаковые данные хранят на нескольких узлах. Если один недоступен, система может обратиться к другой копии, если предусмотрено переключение. Цена репликации — дополнительное место и затраты на передачу и согласование изменений.

Пример и вывод. Историю заказов разделили по месяцам: запрос за сентябрь читает сентябрьскую часть. Затем для каждой части создали копию на другом сервере: при отказе основного узла остаётся второй экземпляр. Разбиение и репликация дополняют друг друга. Репликация также не заменяет резервное копирование: ошибочное удаление может распространиться на все реплики.

4. Почему OLTP и OLAP обычно разделяют архитектурно?

Простыми словами. OLTP обслуживает текущие операции приложения: создание заказа, изменение остатков, запись платежа. OLAP выполняет аналитику: сравнение продаж, расчёт показателей и исследование истории. Эти нагрузки требуют разных условий работы.

Как это работает. Для OLTP характерны многочисленные короткие транзакции с небольшим числом затрагиваемых записей. Важны целостность данных и быстрый предсказуемый ответ. Для OLAP характерны чтение больших массивов, соединения таблиц, группировки и агрегаты. Такой запрос может надолго занять процессор, память и диски, мешая операциям приложения. Поэтому данные передают в отдельный аналитический контур через ETL/ELT или доставку изменений. В аналитическом слое можно подготовить удобную схему, историю и витрины; при этом надо учитывать задержку обновления данных.

Пример и вывод. Покупатель оформляет заказ, а аналитик рассчитывает выручку по всем товарам за три года. На общей базе отчёт может замедлить оформление заказов. В отдельном OLAP-контуре тяжёлый расчёт использует

свои ресурсы. Разделение защищает скорость приложения и позволяет отдельно оптимизировать аналитику; для небольшой нагрузки отдельные системы могут быть избыточны.

5. Чем Data Warehouse отличается от Data Lake?

Простыми словами. Data Warehouse (DWH) — аналитическое хранилище, где данные приведены к согласованному виду для отчётности. Data Lake — озеро данных, где можно сохранять разнообразные исходные материалы для последующей обработки.

Как это работает. В DWH обычно есть определённая схема, очищенные данные, согласованные справочники и история. Это помогает разным отчётам одинаково считать показатели. Data Lake может хранить таблицы, JSON, логи, тексты и изображения. Для исходного слоя характерна схема на чтение: структуру и правила интерпретации применяют при использовании данных. Однако озеро тоже требует каталогизации, контроля качества и доступа; в нём могут находиться и обработанные наборы. Без управления трудно понять, что означает файл и можно ли ему доверять.

Пример и вывод. Магазин сохраняет в Data Lake сырые события сайта, отзывы и изображения товаров. После очистки данные о заказах и клиентах поступают в DWH, а из него — в витрину продаж по дням и категориям. DWH удобен для повторяемой отчётности, Data Lake — для хранения разнообразных данных и новых задач анализа. Они могут использоваться вместе; lakehouse объединяет свойства обоих подходов.

6. В чём различие batch и stream processing?

Простыми словами. Batch processing обрабатывает накопленную порцию данных. Stream processing обрабатывает поступающие события непрерывно, поддерживая результат по мере их появления.

Как это работает. У пакетной задачи есть конечный входной набор: например, все заказы за вчера. Её запускают по расписанию или по запросу и получают итоговый отчёт, витрину или модель. У потока нет заранее

известного конца. Поточковая обработка поддерживает состояние, например сумму событий за последние пять минут. Здесь важны задержка, порядок событий, опоздавшие сообщения, повторы и восстановление после сбоя. Поточковый движок может обрабатывать события по одному или небольшими микропакетами; потоковый подход не означает нулевой задержки.

Пример и вывод. Ночью посчитать выручку за прошедший день — batch. В течение дня обновлять число ошибок сервиса за последние пять минут — stream. Выбор зависит от того, сколько можно ждать между событием и полезным результатом. Если достаточно ежедневного отчёта, постоянная потоковая обработка может неоправданно усложнять систему.

7. Когда целесообразен ETL, а когда ELT?

Простыми словами. В ETL данные сначала извлекают, затем преобразуют и только после этого загружают в целевое хранилище. В ELT извлечённые данные сначала загружают в аналитическую среду, а преобразования выполняют уже внутри неё.

Как это работает. ETL означает Extract → Transform → Load. Он удобен, когда до загрузки надо привести типы, убрать ошибки, согласовать справочники или исключить поля, которые не должны попадать в целевое хранилище. ELT означает Extract → Load → Transform. Он удобен, когда платформа обладает достаточными вычислительными ресурсами и полезно сохранить исходные данные, чтобы применять новые правила обработки без повторного извлечения. В обоих случаях нужны контроль качества, права доступа и воспроизводимые преобразования. Сохранение сырых данных в ELT не делает их автоматически готовыми для отчётов.

Пример и вывод. Если DWH принимает только заказы с проверенными датами и едиными кодами товаров, преобразования можно выполнить заранее по ETL. Если JSON-события сайта сохраняются в исходном слое, а затем из них внутри платформы строятся разные витрины, это ELT. Решение определяется местом преобразований, возможностями платформы и

требованиями к загружаемым данным. Оба подхода могут присутствовать в одной системе.

8. Какую роль в курсе играют PostgreSQL, Spark, Superset и Prometheus?

Простыми словами. Это инструменты разных частей аналитической системы: PostgreSQL хранит структурированные данные, Spark выполняет обработку, Superset показывает аналитические результаты, Prometheus собирает метрики состояния системы.

Как это работает. PostgreSQL — реляционная СУБД: таблицы, SQL, транзакции, ограничения целостности, индексы и аналитические витрины. Spark — вычислительный слой для распределённых преобразований, пакетной и потоковой обработки; PySpark даёт доступ к нему из Python. Superset — BI-инструмент для исследования данных, графиков, фильтров и дашбордов. Prometheus собирает и хранит метрики как временные ряды и позволяет проверять условия для оповещений. В связке курса Grafana визуализирует мониторинговые показатели.

Пример и вывод. Заказы магазина находятся в PostgreSQL. Spark объединяет их с событиями сайта и рассчитывает показатели, результат сохраняется в витрину. Superset показывает продажи по категориям. Prometheus помогает следить за доступностью компонентов, задержками и ошибками через соответствующие метрики. Так соединяются хранение, обработка, бизнес-аналитика и мониторинг инфраструктуры. Spark при этом не заменяет постоянное хранилище, а Superset — обработку и очистку исходных данных.

Лекция №2. Подробные ответы на контрольные вопросы

Тема: «Хранение данных. Модель SQL, NoSQL и анализ запросов».

Вопросы из лекции №2, страница 107. SQL-примеры ниже — учебные; названия таблиц и полей поясняют соответствующую задачу.

1. Почему DDL, DML и TCL отражают разные уровни работы с БД?

Простыми словами. DDL определяет, как устроена база. DML позволяет работать с записями. TCL управляет тем, какие изменения должны быть зафиксированы вместе или отменены.

Как это работает. DDL — Data Definition Language: CREATE, ALTER, DROP создают, изменяют и удаляют объекты базы. Например, задают таблицу, типы столбцов, первичный ключ и ограничения. DML — Data Manipulation Language: INSERT, UPDATE, DELETE добавляют, изменяют и удаляют записи, а SELECT читает данные. В классификации лекции SELECT относится к DML; в других классификациях его выделяют в DQL. TCL — Transaction Control Language: BEGIN, COMMIT, ROLLBACK задают границы транзакции, подтверждают изменения или отменяют их. Это разделение по назначению команд, а не три независимые базы данных.

Пример и вывод. Создать таблицу заказов с обязательным идентификатором клиента — DDL. Записать новый заказ и уменьшить остаток товара — DML. Объединить эти изменения в одну транзакцию — TCL. Приложение должно проверить успешность обоих действий и только затем выполнить COMMIT; при ошибке — ROLLBACK. Одна транзакция сама по себе не заменяет проверку наличия товара и других бизнес-условий. Три уровня отвечают на вопросы: «какова структура?», «что сделать с данными?» и «что сохранить как единую операцию?».

2. Чем оконная функция отличается от GROUP BY?

Простыми словами. GROUP BY объединяет строки в группы и позволяет получить один итог на группу. Оконная функция добавляет каждой строке

показатель по связанным с ней строкам, сохраняя детализацию входа оконного вычисления.

Как это работает. Обычная агрегация `SUM(amount)` с `GROUP BY customer_id` возвращает общую сумму по каждому клиенту. Запись `SUM(amount) OVER (PARTITION BY customer_id)` возвращает сумму клиента рядом с каждым его заказом. Окна удобны для рангов, долей, накопительных сумм и сравнения с предыдущей строкой. Если в запросе уже есть группировка, окно работает над её результатом — оно не возвращает строки, которые были объединены ранее.

Пример и вывод. У клиента А два заказа на 100 и 200 рублей, у клиента В — один на 50 рублей. Группировка вернёт две строки: А — 300, В — 50. Оконная сумма сохранит три заказа и добавит значения 300, 300, 50 соответственно.

```
SELECT order_id, customer_id, amount,  
       SUM(amount) OVER (PARTITION BY customer_id) AS customer_total  
FROM orders  
ORDER BY customer_id, order_id;
```

Если нужен только итог по клиенту, подходит группировка. Если нужно показать каждый заказ вместе с итогом клиента, подходит оконная функция.

3. Зачем в оконной функции нужны PARTITION BY, ORDER BY и frame?

Простыми словами. `PARTITION BY` задаёт, какие строки относятся к одной группе расчёта. `ORDER BY` определяет их последовательность. `Frame` задаёт границы внутри группы для функций, учитывающих рамку окна, например оконной суммы или среднего.

Как это работает. `PARTITION BY customer_id` отделяет заказы одного клиента от заказов других. `ORDER BY order_date, order_id` задаёт порядок заказов; уникальный дополнительный ключ делает последовательность однозначной при совпадающих датах. Рамка `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` включает строки от начала группы до текущей — получается накопительный итог. Рамка `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` включает текущую и до шести

предыдущих строк. Это семь строк, а не обязательно семь календарных дней: для такой интерпретации нужны ровно одна строка на день и отсутствие пропусков дат.

Пример и вывод. Для заказов одного клиента на 100, 200 и 50 рублей следующий запрос даёт накопительные суммы 100, 300 и 350 рублей. У следующего клиента накопление начинается заново.

```
SELECT customer_id, order_id, order_date, amount,  
       SUM(amount) OVER (  
         PARTITION BY customer_id  
         ORDER BY order_date, order_id  
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
       ) AS running_total  
FROM orders  
ORDER BY customer_id, order_date, order_id;
```

Без `PARTITION BY` все строки составляют одну группу. Внутренний `ORDER BY` задаёт порядок вычисления, а окончательный порядок вывода задаёт внешний `ORDER BY`. При наличии сортировки рамка по умолчанию в PostgreSQL включает также строки с одинаковыми значениями ключей сортировки; явный `ROWS` помогает точно выразить построчное накопление. Рамка влияет не на все оконные функции: например, `ROW_NUMBER` нумерует строки группы и не ограничивается указанным `frame`. Уточнения о порядке и рамке: документация PostgreSQL (<https://www.postgresql.org/docs/current/tutorial-window.html>).

4. Почему EXPLAIN ANALYZE может изменить данные при неосторожном использовании?

Простыми словами. `EXPLAIN` показывает предполагаемый план выполнения. `EXPLAIN ANALYZE` действительно запускает запрос, чтобы измерить его работу. Поэтому перед `UPDATE` или `DELETE` это не просто просмотр — изменение или удаление произойдёт.

Как это работает. Для измерения фактического времени, числа строк и повторов выполнения PostgreSQL должен выполнить операции плана. Например, `EXPLAIN ANALYZE DELETE FROM orders` действительно удалит строки, если команда успешно выполнится и изменения будут зафиксированы.

Для изучения обычных транзакционных изменений можно использовать тестовую базу и явную транзакцию с последующим откатом.

```
BEGIN;
```

```
EXPLAIN ANALYZE  
UPDATE orders  
SET status = 'archived'  
WHERE order_date < DATE '2025-01-01';
```

```
ROLLBACK;
```

Пример и вывод. Здесь обновление выполняется и измеряется, после чего ROLLBACK отменяет изменения строк. Но выполнение всё равно создаёт нагрузку и может удерживать блокировки. Кроме того, откат не отменяет абсолютно любые последствия: например, полученные через nextval значения последовательности не возвращаются для повторного использования, а внешние действия пользовательских функций могут быть необратимыми. Даже SELECT может вызвать функцию с побочным эффектом. Перед запуском надо понимать сам запрос; для получения только оценочного плана используется EXPLAIN без ANALYZE. См. EXPLAIN (<https://www.postgresql.org/docs/current/sql-explain.html>) и последовательности PostgreSQL (<https://www.postgresql.org/docs/current/functions-sequence.html>).

5. Что означают Seq Scan, Index Scan, Hash Join, WindowAgg?

Простыми словами. Это названия операций в плане запроса. Они показывают, каким способом PostgreSQL читает записи, соединяет наборы и вычисляет оконные показатели.

Как это работает. Seq Scan — последовательное чтение таблицы с проверкой условия, если оно есть. Index Scan — поиск через индекс с обращением к строкам таблицы. Hash Join — соединение по условию равенства: по одному входному набору строится хеш-таблица, а строки другого ищут в ней совпадения по ключу. WindowAgg — вычисление оконных функций над входными строками. Перед ним часто появляется Sort, если для окна нужен порядок, который входной поток ещё не обеспечивает.

Пример и вывод. Найти один заказ по индексированному идентификатору может быть удобно через Index Scan. Для суммы по почти всей истории заказов Seq Scan может оказаться дешевле множества обращений через индекс. При соединении большого набора заказов с клиентами возможен Hash Join; при добавлении накопительной суммы — WindowAgg. Конкретный план зависит от объёма данных, условий, индексов, статистики и настроек. Нельзя объявлять запрос плохим только из-за Seq Scan или хорошим только из-за индекса. В плане сравнивают ожидаемые и фактические строки, время и повторения; cost — условная оценка стоимости, а не миллисекунды. Подробнее: чтение планов PostgreSQL (<https://www.postgresql.org/docs/current/using-explain.html>).

6. Почему NoSQL нельзя сводить к одной технологии?

Простыми словами. NoSQL — общее название семейства систем с разными моделями хранения и доступа. Это не одна СУБД и не единый способ работы с данными.

Как это работает. В key-value основная операция — получить значение по ключу. В document данные представлены вложенными документами. Wide-column ориентирована на распределённые записи и доступ по заранее спроектированным ключам. Graph хранит объекты и связи, позволяя выполнять обходы. В лекции также рассматриваются поисковые и временные модели: поиск по тексту и работа с метриками во времени. У конкретных систем различаются индексы, транзакции, согласованность, масштабирование и языки запросов. Поэтому нельзя утверждать, что любая NoSQL-база не имеет схемы, не поддерживает транзакции или всегда работает быстрее SQL-базы.

Пример и вывод. Получить сессию по идентификатору, прочитать карточку товара с вложенными характеристиками и найти цепочку связей между пользователями — три разные задачи. Одна система может поддерживать несколько моделей, но это не устраняет различия между ними. Выбирать следует по данным, запросам и ограничениям, а не по одному слову

NoSQL. Выражение «not only SQL» подчёркивает, что такие системы дополняют реляционный подход.

7. Как выбрать между document, key-value, wide-column и graph model?

Простыми словами. Сначала нужно сформулировать, какие запросы должны выполняться быстро. Затем выбрать представление данных и ключи, которые делают эти запросы удобными.

Как это работает. Document подходит, когда приложение читает и изменяет вложенный объект: карточку товара, профиль, заказ с позициями. Key-value подходит, когда известен точный ключ и нужно получить связанное значение: сессию или корзину пользователя. Wide-column полезна для интенсивной записи событий и чтения истории конкретного объекта по диапазону времени; ключ распределения и порядок внутри раздела проектируют заранее. Graph подходит, когда главное — связи, пути и соседства: рекомендации или поиск общих связей между объектами. Дополнительно проверяют размер данных, частоту чтения и записи, требования к согласованности и транзакциям, возможность появления новых запросов и стоимость сопровождения.

Пример и вывод. У интернет-магазина каталог товаров с разными характеристиками может использовать document; получение корзины по user_id — key-value; история событий устройства — wide-column; поиск товаров через связи «пользователь — покупка — товар» — graph. Для истории событий ключ можно строить по устройству и периоду, чтобы данные одного активного устройства не росли в одном разделе бесконечно. Конкретное разбиение надо проверять на распределении нагрузки. Wide-column не следует путать с колоночным хранением аналитической СУБД: это разные понятия. Несколько моделей в одной архитектуре допустимы, но каждая дополнительная система должна оправдывать сложность интеграции и обслуживания.

8. Почему NoSQL-хранилище часто не является конечным BI-слоем?

Простыми словами. Операционное NoSQL-хранилище обычно организовано для запросов приложения. BI нужны стабильные показатели, понятные поля и удобные сравнения по всей истории. Подготовка данных для этих задач часто требует отдельного слоя.

Как это работает. В источнике могут быть вложенные документы, неодинаковые наборы полей, дубли и данные, распределённые под конкретные ключи. Для BI нужно согласовать типы и справочники, обработать вложенность, определить детализацию и правила вычисления метрик. Кроме того, тяжёлый отчёт по всем данным может конкурировать с пользовательскими запросами. Поэтому данные выгружают или передают изменениями в Data Lake, DWH либо другую аналитическую среду, обрабатывают и формируют витрину. Витрина даёт измерения, например дату и категорию, и меры, например сумму продаж и число заказов.

Пример и вывод. Заказ хранится документом с массивом товарных позиций. Для отчёта по категориям позиции превращают в отдельные строки, добавляют категорию товара и рассчитывают выручку по согласованным правилам. При этом нельзя суммировать полную стоимость заказа после её повторения в каждой позиции — получится завышение. Подготовленную витрину подключают к BI-инструменту. NoSQL может поддерживать собственные агрегации и непосредственное подключение аналитики; отдельный слой нужен тогда, когда его оправдывают нагрузка, структура данных и требования к качеству показателей.

Источники

Технологии и инструментарий анализа больших данных. Лекция №1. Предоставленный PDF, контрольные вопросы на странице 70.

Технологии и инструментарий анализа больших данных. Лекция №2. Предоставленный PDF, контрольные вопросы на странице 107.